

PROJECT REPORT

(Project Term March-June 2026)

(Resume Previewer using DevOps Tools and CI/CD Pipeline)

Submitted by

(Buchupalle Purna Tejeshwara Reddy) Registration Number :12401711

Course Code : INT345

Under the Guidance of

(Ms. Jatinder Kaur, Assistant Professor)

School of Computer Science and Engineering



Transforming Education Transforming India

DECLARATION

I hereby declare that the project titled “Resume Previewer using DevOps Tools and CI/CD Pipeline” is an original piece of work carried out by me as part of the requirements for the award of Bachelor of Technology in Computer Science and Engineering from Lovely Professional University, Phagwara.

This project was completed under the guidance of Jatinder Kaur, Assistant Professor, during the period March 2026 to May 2026.

The work presented in this report is based on my own implementation, testing, deployment, and documentation work. All tools, technologies, and resources used in this project have been properly studied and implemented for educational purposes. No part of this project report has been submitted earlier for any other degree, diploma, or academic award.

Signature: 

Name: Buchupalle Purna Tejeshwara Reddy

Registration Number: 12401711

Date: May 2026

CERTIFICATE

This is to certify that the project titled “Resume Previewer using DevOps Tools and CI/CD Pipeline” has been successfully carried out by Buchupalle Purna Tejeshwara Reddy under my guidance and supervision in partial fulfilment of the requirements for the award of Bachelor of Technology in Computer Science and Engineering from Lovely Professional University, Phagwara.

The work presented in this report is genuine and has been completed by the student independently using appropriate tools and technologies related to DevOps, Docker, Jenkins, GitHub Actions, and cloud deployment platforms.

To the best of my knowledge, this project has not been submitted elsewhere for any other degree or academic qualification.

Name of Faculty Mentor: Jatinder Kaur

Designation: Assistant Professor

School of Computer Science and Engineering

Lovely Professional University, Phagwara, Punjab

Date: _____

Signature: _____

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my faculty mentor, Jatinder Kaur, Assistant Professor, for her continuous guidance, encouragement, and valuable suggestions throughout the development of this project. Her support helped me understand the practical implementation of DevOps concepts and complete the project successfully.

I am thankful to Lovely Professional University for providing the learning environment, technical resources, and platform required to work on a real-time industry-oriented project. The knowledge gained from the university courses and lab sessions played an important role during the development and deployment process.

This project helped me gain practical experience in technologies such as GitHub, Jenkins, Docker, CI/CD Pipelines, Render, and Vercel deployment platforms. Working on this project improved my understanding of software deployment automation and modern DevOps workflows.

Finally, I would like to thank my family and friends for their motivation, support, and encouragement throughout the project development and report preparation process.

Buchupalle Purna Tejeshwara Reddy

B.Tech Computer Science and Engineering

Lovely Professional University, Phagwara

Table of Contents

1. Introduction	
1.1 Introduction to the Project	
1.2 Objectives of the Project	
2. Scope of the Study (Problem Statement)	
2.1 Problem Statement	
2.2 Scope of the Study	
2.3 Need for Automation	
3. Existing System	
3.1 Introduction	
3.2 Existing Software	
3.3 DFD for Present System	
3.4 Problems in Existing System	
3.5 What's New in the System to be Developed	
3.6 Proposed System	
4. Problem Analysis	
4.1 Product Definition	
4.2 Feasibility Analysis	
4.3 Project Plan	
4.4 Methodology	
5. Software Requirement Analysis	
5.1 Introduction	
5.2 General Description	
5.3 Specific Requirements	
5.4 Functional Requirements	
5.5 Non-Functional Requirements	
5.6 Tools and Technologies Used	
6. Design	
6.1 System Design	
6.2 Design Notations	
6.3 Detailed Design	
6.4 Architecture Diagram	
6.5 Working Flow Diagram	
6.6 Flowcharts	
6.7 Pseudo Code	
7. Testing	
7.1 Functional Testing	
7.2 Structural Testing	
7.3 Levels of Testing	
7.4 Testing the Project	
7.5 Test Results	
8. Implementation	
8.1 Implementation of the Project	
8.2 Jenkins CI/CD Pipeline Setup	
8.3 Docker Containerization	

8.4 GitHub Actions Workflow	
8.5 Backend Deployment on Render	
8.6 Frontend Deployment on Vercel	
8.7 Conversion Plan	
8.8 Post-Implementation and Software Maintenance	
9. Project Legacy	
9.1 Current Status of the Project	
9.2 Remaining Areas of Concern	
9.3 Technical and Managerial Lessons Learnt	
9.4 Future Enhancements	
9.5 Expected Outcome	
9.6 Project Legacy	
10. Source Code / System Snapshots	
10.1 GitHub Repository Screenshots	
10.2 Jenkins Pipeline Screenshots	
10.3 Docker Container Screenshots	
10.4 Render Deployment Screenshots	
10.5 Vercel Deployment Screenshots	
10.6 Important Source Code Snippets	
11. Conclusion	
12. Bibliography	

CHAPTER 1 INTRODUCTION

Modern software development mainly focuses on automation, fast deployment, and easy maintenance. Traditional deployment methods require manual setup and testing, which takes more time and effort. To solve these problems, DevOps tools and CI/CD pipelines are widely used in the software industry.

This project, “Resume Previewer using DevOps Tools and CI/CD Pipeline”, was developed to understand and implement real-time DevOps workflow. The project includes frontend and backend services connected through APIs. Docker was used for containerization, Jenkins and GitHub Actions were used for automation, and cloud platforms such as Render and Vercel were used for deployment.

The project helps in understanding:

- ❖ Continuous Integration and Continuous Deployment (CI/CD)
- ❖ Docker containerization
- ❖ Jenkins pipeline automation
- ❖ GitHub workflow
- ❖ Cloud deployment process

The main goal of this project is to automate the software build and deployment process and reduce manual work.

1.1 Introduction to the Project

The Resume Previewer project is a simple web application developed using DevOps practices and deployment automation tools. The frontend provides the user interface, while the backend handles application processing.

In this project:

- ❖ GitHub was used for version control
- ❖ Jenkins was used for CI/CD automation
- ❖ Docker was used for creating containers
- ❖ Docker Hub was used for storing images
- ❖ Render was used for backend deployment
- ❖ Vercel was used for frontend deployment

Whenever code changes are pushed to GitHub, the Jenkins pipeline automatically builds and deploys the project.

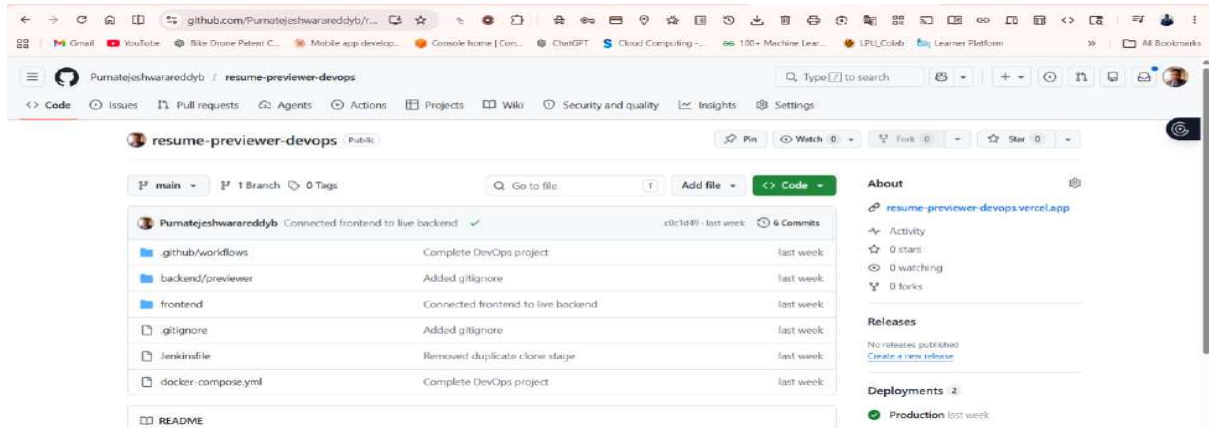
This project gives practical knowledge about real-world DevOps implementation and deployment workflow.

1.2 Objectives of the Project

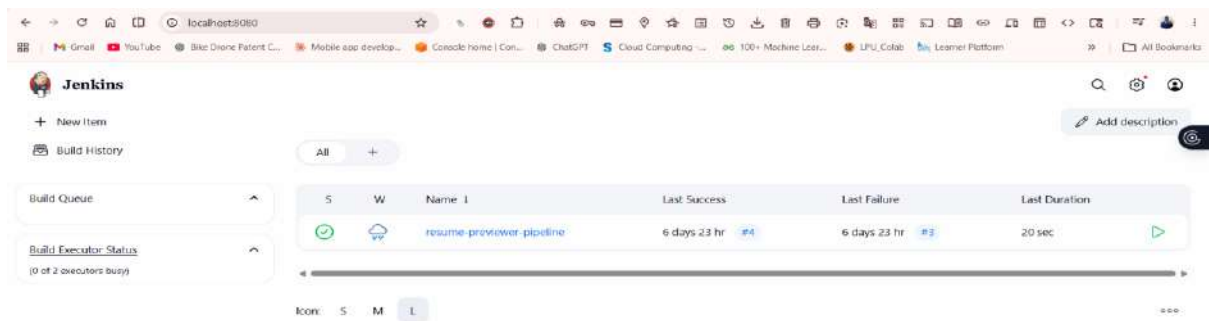
The main objectives of this project are:

- ❖ To develop a Resume Previewer web application.
- ❖ To understand DevOps tools and workflow.
- ❖ To implement CI/CD pipeline automation.

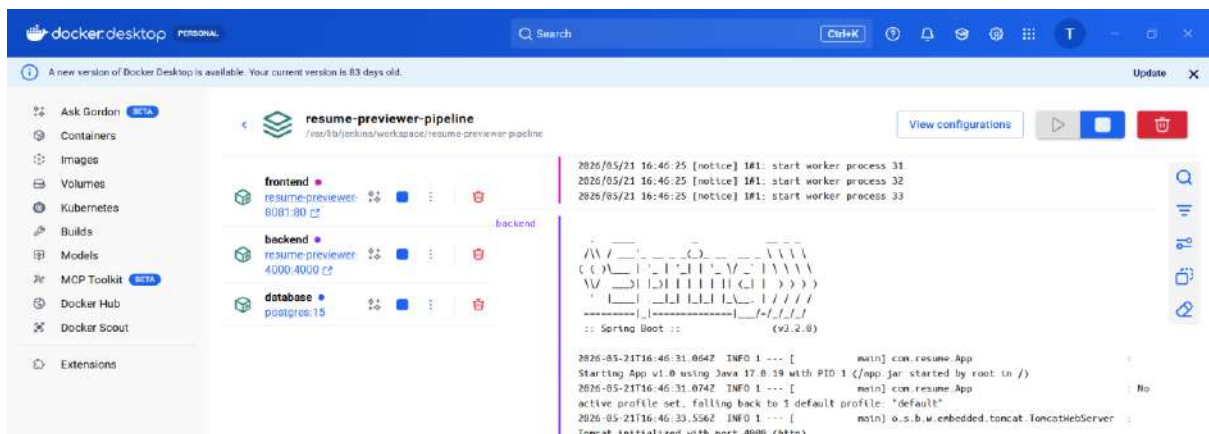
- ❖ To use Docker for application containerization.
- ❖ To integrate Jenkins with GitHub.
- ❖ To deploy applications on cloud platforms.
- ❖ To reduce manual deployment effort.
- ❖ To gain practical knowledge of software deployment and automation.



GitHub Repository



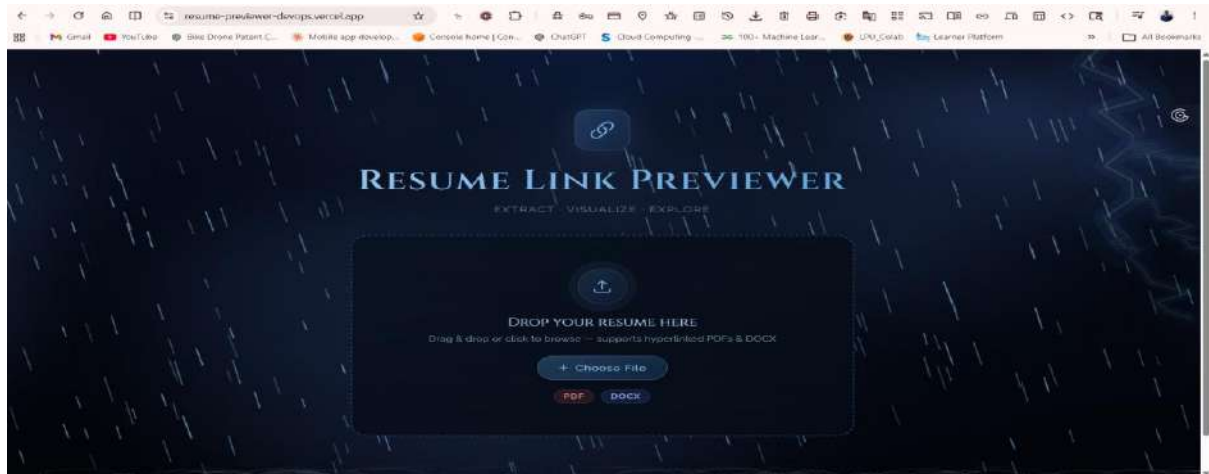
Jenkins Dashboard Screenshot



* Docker Containers Running Screenshot



* Backend Deployment URL Screenshot



* Frontend Deployment Screenshot

2. Scope of the Study (Problem Statement)

2.1 Problem Statement

Many small software projects are developed manually without a proper deployment and automation process. Developers often build applications on their local systems, but deploying, testing, and maintaining them becomes difficult. Manual deployment can lead to errors, version mismatch, downtime, and extra development time.

In the Resume Previewer project, there was a need to automate the software development lifecycle using DevOps practices. The system required automatic code integration, containerization, deployment, and monitoring using modern DevOps tools.

The main problem addressed in this project is how to simplify application deployment and make the development process faster, reliable, and easy to manage using CI/CD pipelines and Docker-based deployment.

2.2 Scope of the Study

The scope of this project is limited to designing and implementing a simple DevOps-based Resume Previewer application with automated deployment support.

The project includes:

- ❖ Frontend development using HTML
- ❖ Backend development using Spring Boot
- ❖ Containerization using Docker
- ❖ CI/CD pipeline using Jenkins
- ❖ Source code management using Git and GitHub
- ❖ Cloud deployment using Render and Vercel
- ❖ Image storage and distribution using Docker Hub

The project demonstrates how DevOps tools can be integrated into a real-world application workflow from development to deployment.

The system is designed mainly for learning and academic purposes and can be extended further in the future with additional features such as database integration, authentication, monitoring, and scaling.

2.3 Need for Automation

In traditional software development, developers manually build, test, and deploy applications. This process takes more time and increases the chances of human error. Every small update requires repeating multiple manual steps.

Automation is needed because it:

- ❖ Reduces manual work
- ❖ Saves development time
- ❖ Improves deployment speed
- ❖ Reduces configuration errors
- ❖ Ensures consistency in application deployment
- ❖ Helps in faster testing and integration
- ❖ Simplifies software maintenance

In this project, automation is achieved using Jenkins CI/CD pipelines and Docker containers. Whenever code is pushed to GitHub, Jenkins automatically pulls the latest code, builds the application, creates Docker images, and deploys the containers.

3. Existing System

3.1 Introduction

Before DevOps practices became popular, most software applications were developed and deployed manually. Developers used separate environments for coding, testing, and deployment, which created many compatibility and maintenance issues.

Applications were often difficult to update and required manual server configuration each time a new version was released.

3.2 Existing Software

Traditional deployment systems mainly use:

- ❖ Manual FTP uploads
- ❖ Local server deployment
- ❖ Manual testing methods
- ❖ Static hosting without automation
- ❖ Direct server configuration

These methods work for small applications but become difficult to manage when projects grow larger.

Some existing tools used in older deployment systems include:

- ❖ XAMPP/WAMP local servers
- ❖ Manual Apache or Nginx configuration
- ❖ Basic hosting platforms without CI/CD support

3.3 DFD for Present System

Level 0 DFD (Existing Manual System)



Figure 3.1: DFD for Existing Manual System

3.3 Problems in Existing System

The existing manual system has several limitations:

- ❖ Deployment process is slow
- ❖ Manual configuration causes errors
- ❖ Difficult to maintain application versions
- ❖ No automatic testing or integration
- ❖ Updating applications takes extra effort
- ❖ Deployment depends heavily on developer actions
- ❖ Difficult to scale and manage multiple services

These problems reduce productivity and increase maintenance complexity.

3.5 What's New in the System to be Developed

The proposed system introduces DevOps automation into the application workflow.

New features added in the developed system include:

- ❖ Automated CI/CD pipeline using Jenkins
- ❖ Docker-based containerized deployment
- ❖ GitHub integration for version control
- ❖ Cloud deployment using Render and Vercel
- ❖ Docker Hub integration for image hosting
- ❖ Faster deployment and testing process
- ❖ Improved reliability and consistency

The new system reduces manual work and provides a modern software deployment workflow.

3.6 Proposed System

The proposed system is a DevOps-enabled Resume Previewer application that automates development, build, testing, and deployment processes.

The workflow of the proposed system is:

1. Developer pushes code to GitHub
2. Jenkins detects code changes
3. Jenkins builds the backend application using Maven
4. Docker images are created automatically
5. Images are pushed to Docker Hub
6. Frontend is deployed using Vercel
7. Backend is deployed using Render
8. Users access the deployed application online

Proposed System Architecture Diagram

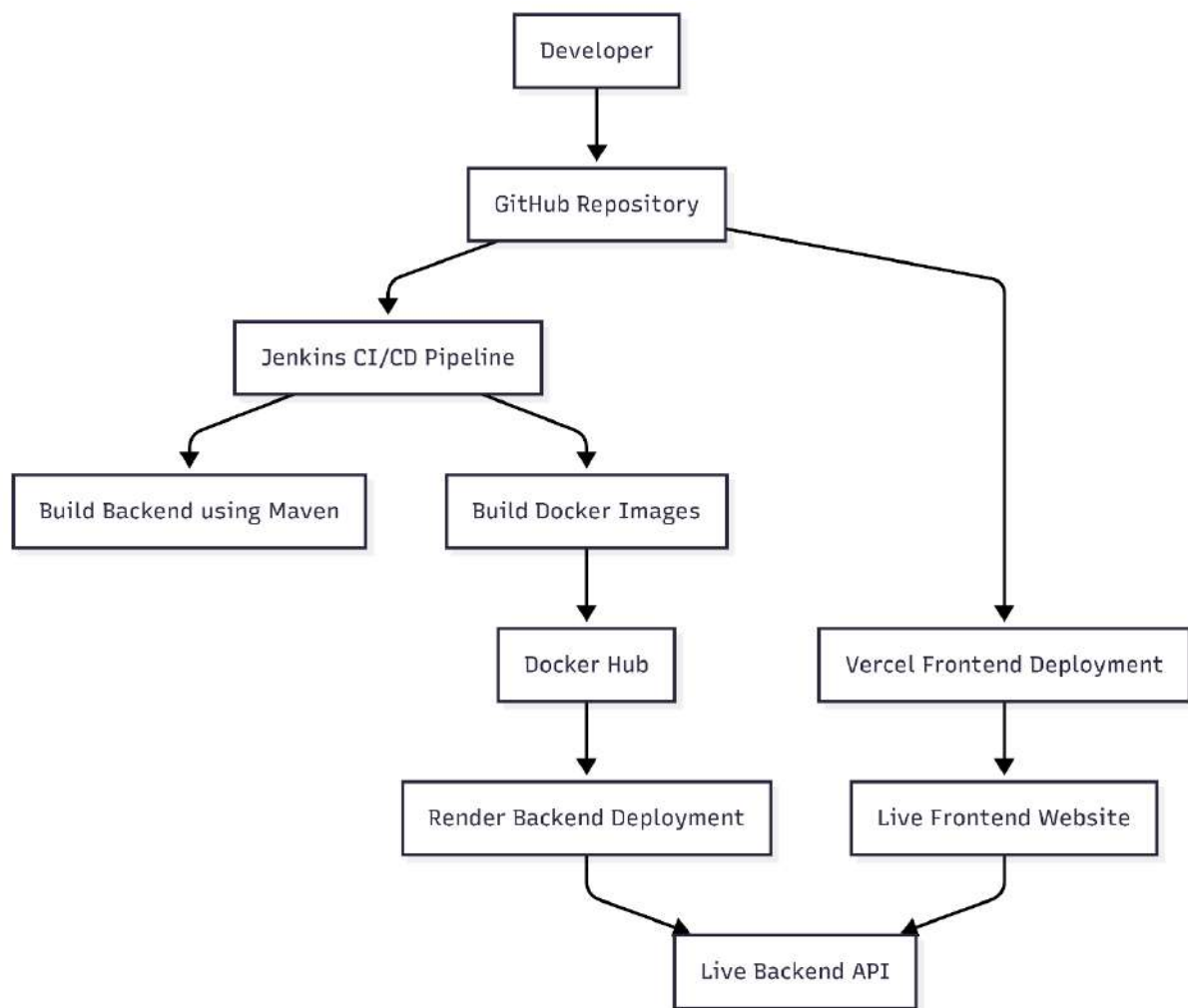


Figure 3.2: Proposed System Architecture

4. Problem Analysis

4.1 Product Definition

The Resume Previewer is a DevOps-based web application developed to demonstrate automated software deployment using modern DevOps tools and practices. The system consists of a frontend interface, backend service, Docker containers, Jenkins CI/CD pipeline, GitHub repository integration, and cloud deployment platforms.

The main purpose of the project is to reduce manual deployment effort and provide an automated workflow for building, testing, containerizing, and deploying applications.

The project also demonstrates how Docker and CI/CD pipelines can simplify application management and deployment in real-world software development.

4.2 Feasibility Analysis

1. Technical Feasibility

The project is technically feasible because all required technologies such as Docker, Jenkins, GitHub, Render, and Vercel are freely available and supported on Linux systems.

The frontend and backend applications were successfully containerized and deployed using Docker containers.

2. Economic Feasibility

The project is economically feasible because most of the tools used are open-source or provide free-tier services.

No additional hardware cost was required since the project was developed using a personal laptop and cloud free-tier platforms.

3. Operational Feasibility

The system is easy to operate and maintain. Developers can push code changes to GitHub, and the CI/CD pipeline automatically handles build and deployment processes.

The deployment workflow reduces manual intervention and improves development efficiency.

4.3 Project Plan

The project was completed in multiple phases.

Phase	Description
Phase 1	Requirement gathering and project planning
Phase 2	Frontend and backend development
Phase 3	Docker container creation
Phase 4	Jenkins CI/CD pipeline setup
Phase 5	GitHub integration
Phase 6	Docker Hub image deployment
Phase 7	Cloud deployment using Render and Vercel
Phase 8	Testing and documentation

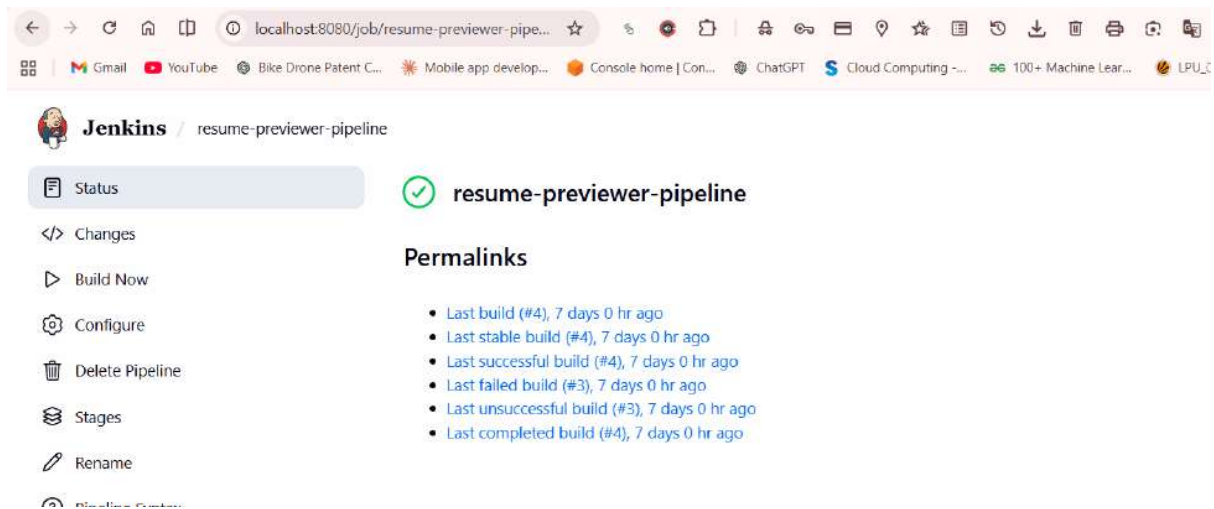


Figure 4.1: Jenkins Pipeline Build Success

4.4 Methodology

The project follows a simple DevOps-based development methodology.

The development process includes:

1. Writing source code for frontend and backend
2. Managing source code using Git and GitHub
3. Building backend using Maven
4. Creating Docker containers
5. Automating deployment using Jenkins CI/CD
6. Pushing Docker images to Docker Hub
7. Deploying frontend and backend to cloud platforms
8. Testing the deployed application

Methodology Flow Diagram

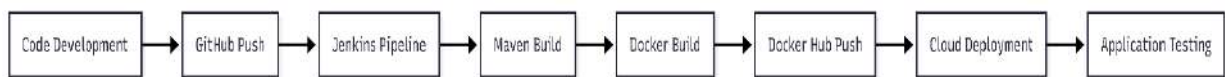


Figure 4.2: Project Methodology Flow

5. Software Requirement Analysis

5.1 Introduction

Software Requirement Analysis is the process of identifying system requirements needed for successful project development.

This phase helps in understanding system functionality, user requirements, tools needed, and deployment workflow before implementation.

The Resume Previewer project requires both software development and DevOps automation components.

5.2 General Description

The Resume Previewer system is a web-based DevOps application consisting of:

- ❖ Frontend web interface
- ❖ Backend Spring Boot service
- ❖ Docker container environment
- ❖ Jenkins CI/CD pipeline
- ❖ GitHub repository integration
- ❖ Cloud deployment services

The system automates the software deployment lifecycle from source code integration to final deployment.

5.3 Specific Requirements

Hardware Requirements

- ❖ Laptop/Desktop
- ❖ Minimum 4 GB RAM
- ❖ Intel/AMD Processor
- ❖ Stable internet connection

Software Requirements

- ❖ Ubuntu/WSL Linux Environment
- ❖ Docker
- ❖ Jenkins
- ❖ Git
- ❖ Java 17
- ❖ Maven
- ❖ VS Code
- ❖ GitHub Account
- ❖ Docker Hub Account

5.4 Functional Requirements

The system should be able to:

- ❖ Upload and manage source code using GitHub
- ❖ Automatically trigger Jenkins pipelines
- ❖ Build backend applications using Maven
- ❖ Create Docker images
- ❖ Push Docker images to Docker Hub
- ❖ Deploy frontend and backend applications
- ❖ Provide live application access to users

5.5 Non-Functional Requirements

The system should provide:

- ❖ Fast deployment process
- ❖ Reliable CI/CD workflow
- ❖ Easy maintainability
- ❖ Platform independence using Docker
- ❖ Secure source code management
- ❖ High availability of deployed services

5.6 Tools and Technologies Used

Tool/Technology	Purpose
HTML	Frontend development
Spring Boot	Backend development
Docker	Containerization
Jenkins	CI/CD automation
Git	Version control
GitHub	Source code hosting
Docker Hub	Docker image hosting
Render	Backend deployment
Vercel	Frontend deployment
Maven	Backend build management
Ubuntu/WSL	Development environment

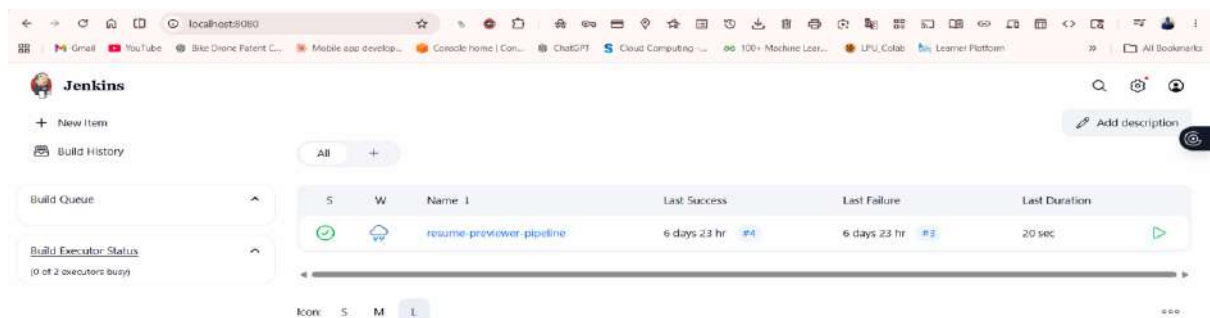


Figure 5.1: Jenkins Dashboard

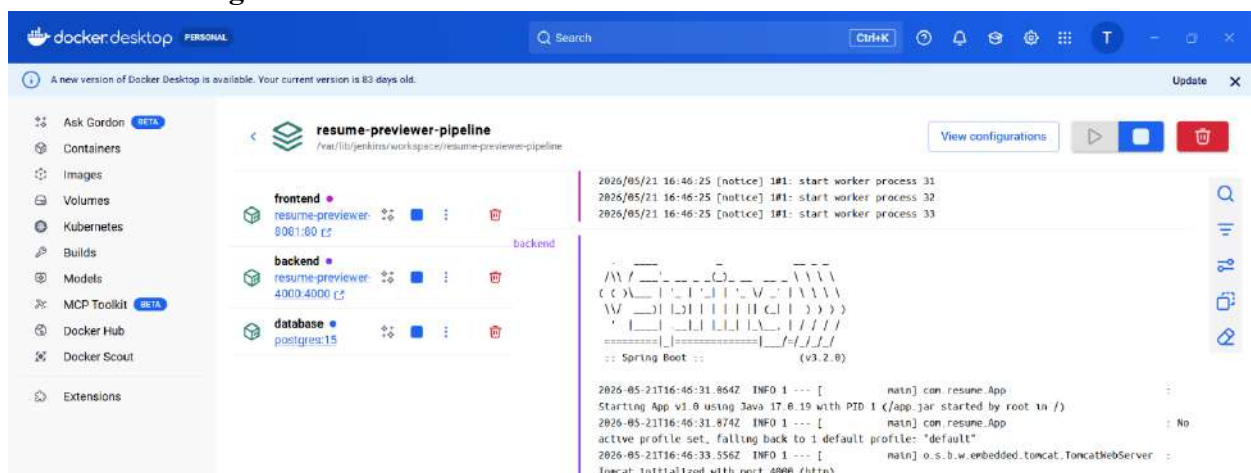


Figure 5.2: Docker Containers

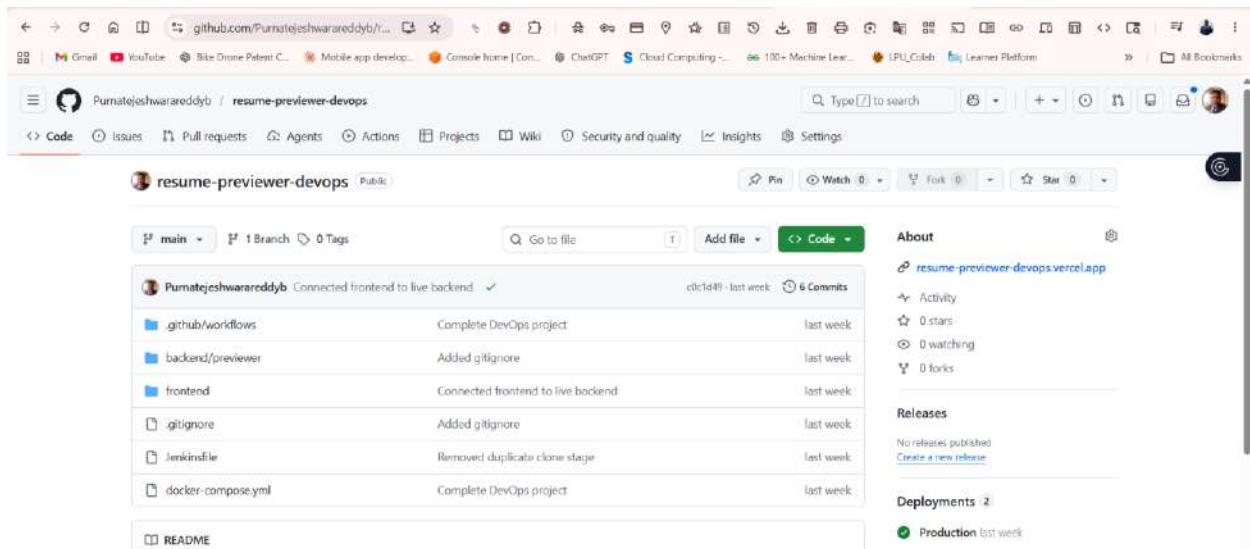


Figure 5.3: GitHub Repository

6. Design

6.1 System Design

The system is designed using a client-server architecture.

The frontend handles user interaction, while the backend processes application requests. Jenkins automates build and deployment tasks. Docker containers ensure consistent deployment across environments.

The frontend and backend are deployed separately using cloud hosting platforms.

6.2 Design Notations

The project uses different design notations such as:

- ❖ Flowcharts
- ❖ Architecture diagrams
- ❖ DFD diagrams
- ❖ CI/CD workflow diagrams

These diagrams help in understanding the system workflow and deployment process clearly.

6.3 Detailed Design

Frontend Module

- ❖ Built using HTML
- ❖ Displays Resume Previewer interface
- ❖ Communicates with backend API

Backend Module

- ❖ Developed using Spring Boot
- ❖ Handles backend processing
- ❖ Runs inside Docker container

CI/CD Module

- ❖ Managed using Jenkins
- ❖ Automates build and deployment

Deployment Module

- ❖ Docker Hub stores images
- ❖ Render hosts backend
- ❖ Vercel hosts frontend

6.4 Architecture Diagram

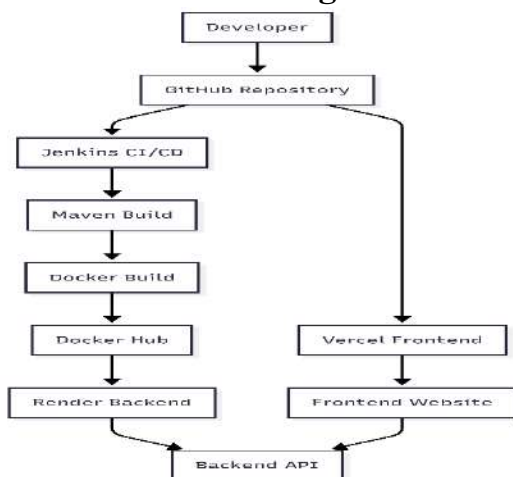


Figure 6.1: System Architecture Diagram

6.5 Working Flow Diagram



Figure 6.2: Working Flow Diagram

6.6 Flowcharts

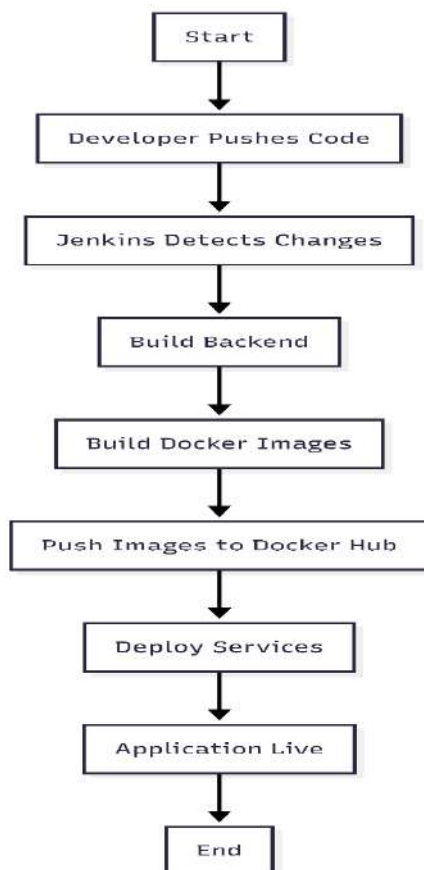


Figure 6.3: Deployment Flowchart

6.7 Pseudo Code

START

Write application code

Push code to GitHub repository

Jenkins detects repository changes

Build backend using Maven

Create Docker images

Push Docker images to Docker Hub

Deploy frontend to Vercel

Deploy backend to Render

Run application containers

Access live application

END



Resume Previewer Backend Running 🚀

Figure 6.4: Backend Deployment

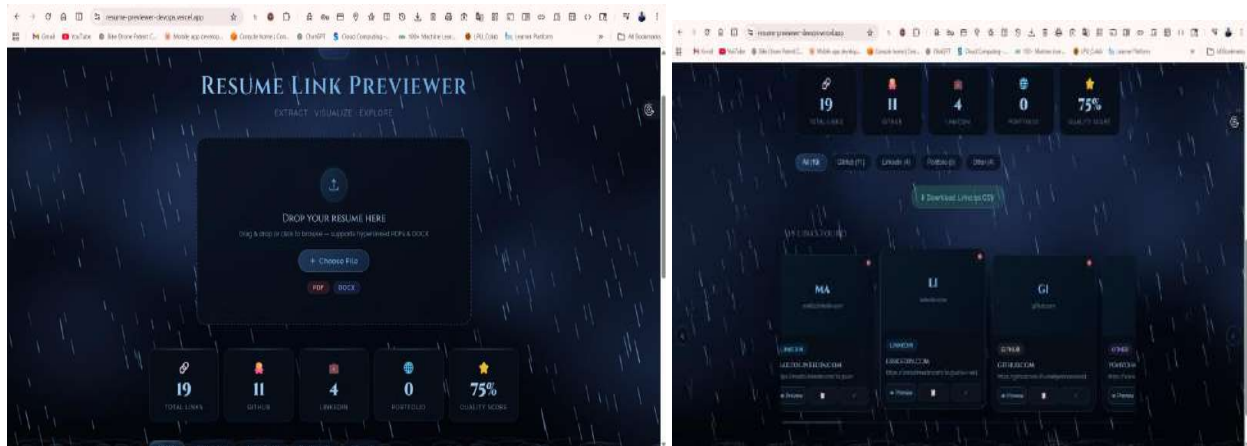


Figure 6.5: Frontend Deployment

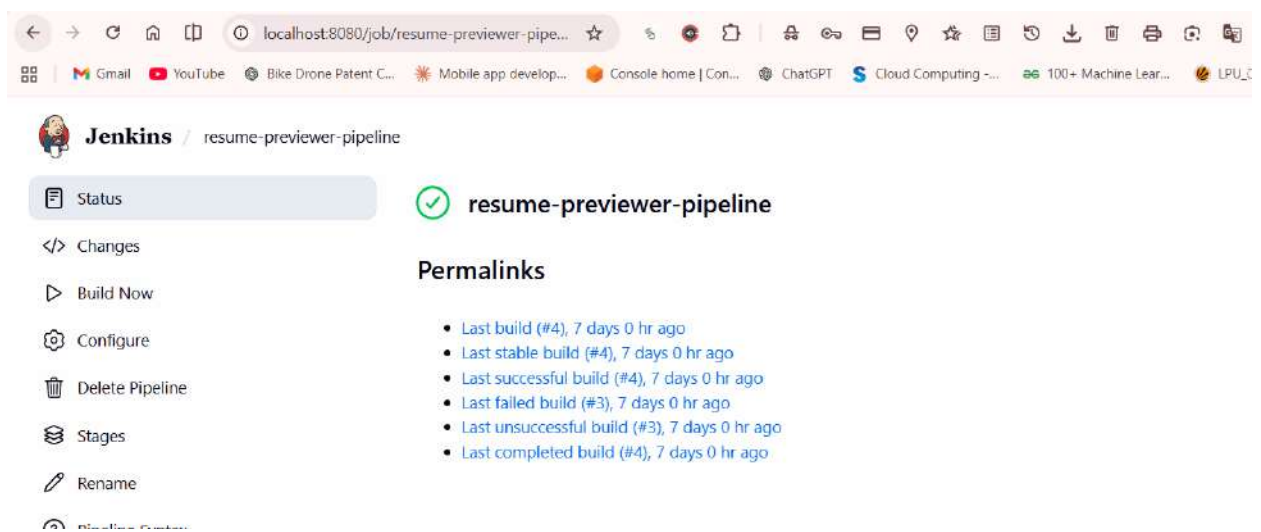


Figure 6.6: Successful Jenkins Build

7. Testing

7.1 Functional Testing

Functional testing was performed to verify whether all modules of the project were working correctly according to the requirements.

The following functionalities were tested:

- ❖ Frontend loading correctly
- ❖ Backend API responding properly
- ❖ Jenkins pipeline execution
- ❖ Docker container creation
- ❖ GitHub repository integration
- ❖ Docker Hub image push
- ❖ Render backend deployment
- ❖ Vercel frontend deployment

All major functions worked successfully after deployment.

7.2 Structural Testing

Structural testing was performed to check the internal workflow and integration between different modules of the system.

The testing included:

- ❖ Backend service communication
- ❖ Docker networking
- ❖ Jenkins stage execution
- ❖ GitHub integration workflow
- ❖ Deployment process validation

The internal system workflow executed correctly without major errors after configuration fixes.

7.3 Levels of Testing

1. Unit Testing

Individual components such as frontend and backend modules were tested separately.

2. Integration Testing

Integration testing was performed between:

- ❖ GitHub and Jenkins
- ❖ Jenkins and Docker
- ❖ Docker Hub and deployment services

3. System Testing

The complete application workflow was tested from code push to live deployment.

4. Deployment Testing

The deployed frontend and backend services were tested using live URLs on Render and Vercel.

7.4 Testing the Project

The project was tested in multiple stages during development.

Test Case	Expected Result	Status
Frontend loading	Webpage opens successfully	Passed
Backend API response	Backend responds correctly	Passed

Test Case	Expected Result	Status
Jenkins build	Pipeline executes successfully	Passed
Docker image build	Docker images created	Passed
Docker Hub push	Images uploaded successfully	Passed
Render deployment	Backend deployed online	Passed
Vercel deployment	Frontend deployed online	Passed

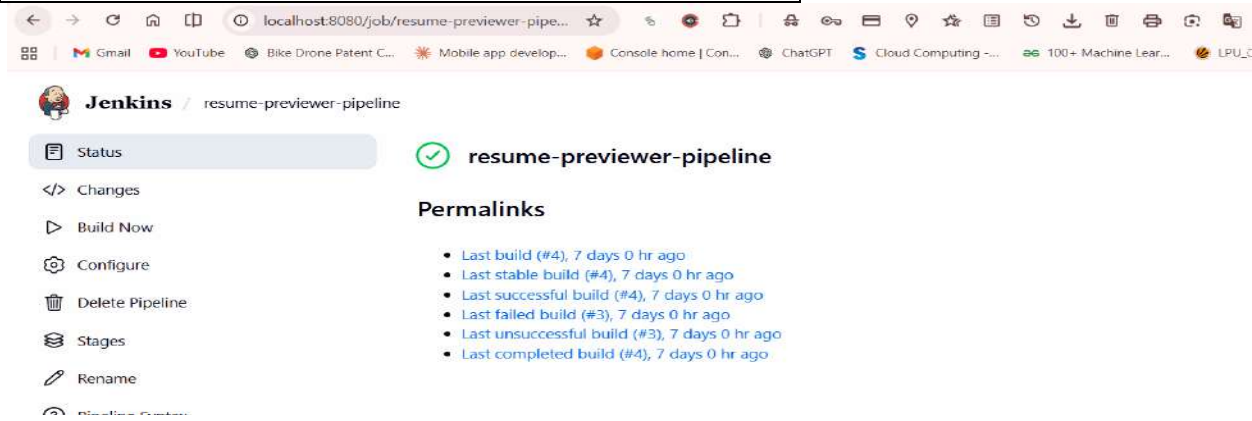


Figure 7.1: Jenkins Successful Pipeline

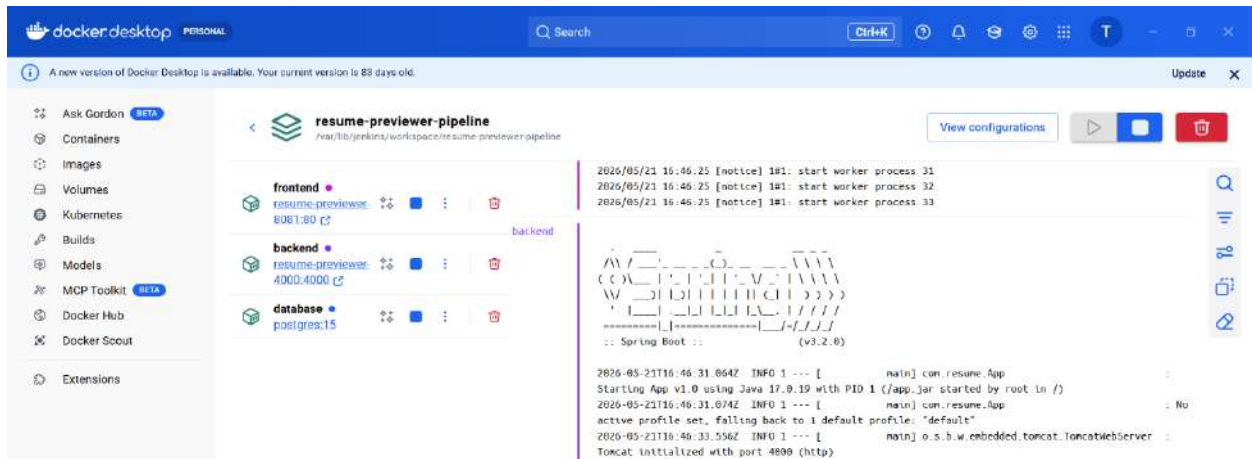


Figure 7.2: Running Docker Containers



Figure 7.3: Backend Deployment on Render

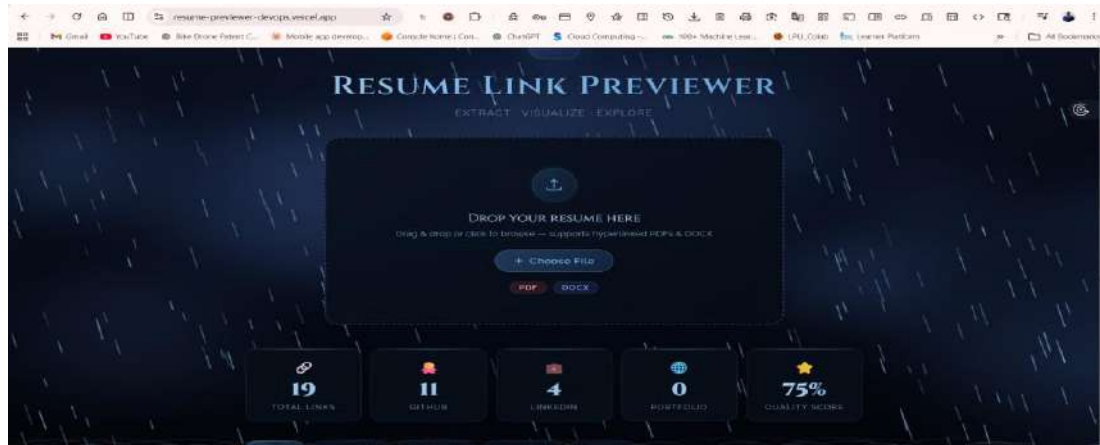


Figure 7.4: Frontend Deployment on Vercel

7.5 Test Results

The testing results showed that the project worked successfully in both local and cloud environments.

The CI/CD pipeline automated the deployment workflow correctly. Docker containers were created successfully, and cloud deployment platforms hosted the application without major issues.

The final system achieved the project objectives of automation, deployment consistency, and simplified DevOps workflow management.

8. Implementation

8.1 Implementation of the Project

The implementation phase focused on building the complete DevOps workflow for the Resume Previewer application.

The project was implemented using:

- ❖ HTML frontend
- ❖ Spring Boot backend
- ❖ Docker containers
- ❖ Jenkins CI/CD pipeline
- ❖ GitHub version control
- ❖ Docker Hub image storage
- ❖ Render and Vercel cloud deployment

The application was first tested locally and later deployed online.

8.2 Jenkins CI/CD Pipeline Setup

Jenkins was used to automate the Continuous Integration and Continuous Deployment process.

The pipeline performs the following tasks automatically:

1. Pull latest code from GitHub
2. Build backend using Maven
3. Build Docker images
4. Deploy containers

The Jenkins pipeline reduced manual deployment work and improved development efficiency.

Jenkins Pipeline Flow



Figure 8.1: Jenkins CI/CD Pipeline

8.3 Docker Containerization

Docker was used to containerize both frontend and backend applications.

Separate Dockerfiles were created for:

- ❖ Frontend service
- ❖ Backend service

Docker Compose was used to manage multiple containers together.

Containerization helped maintain consistency across development and deployment environments.



Figure 8.2: Docker Containers Running

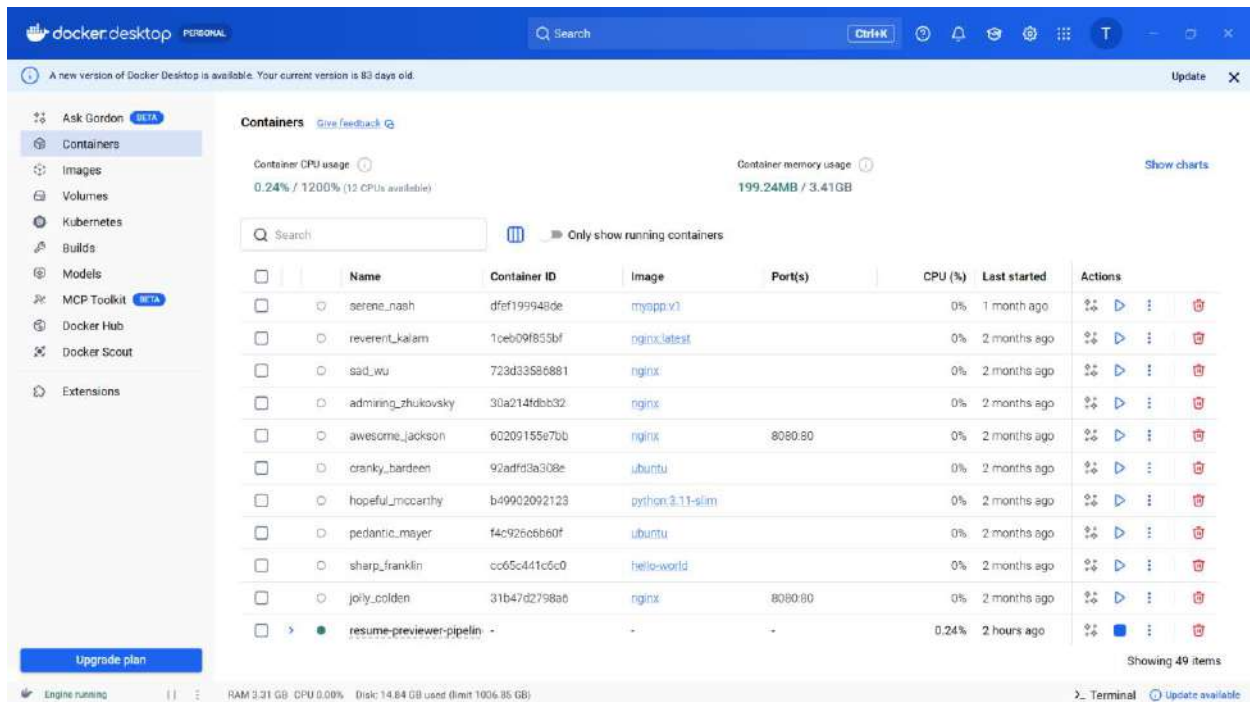


Figure 8.3: Docker Compose Deployment

8.4 GitHub Actions Workflow

GitHub was used as the main source code repository for version control and collaboration.

The project repository stored:

- ❖ Frontend source code
- ❖ Backend source code
- ❖ Docker configuration files
- ❖ Jenkins pipeline files

GitHub integration helped Jenkins automatically detect source code changes during CI/CD execution.

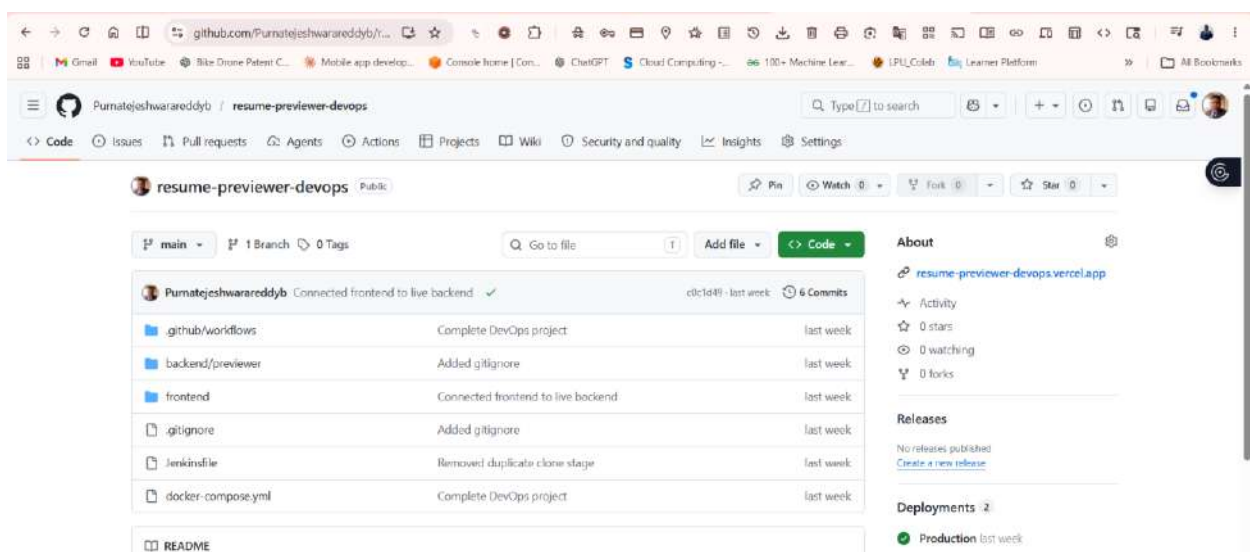


Figure 8.4: GitHub Repository Integration

8.5 Backend Deployment on Render

The backend Docker image was deployed using Render cloud services.

The backend service was connected with Docker Hub images for deployment.

The deployed backend URL successfully responded with:

Resume Previewer Backend Running 🚀

The backend deployment verified successful cloud hosting of the application.

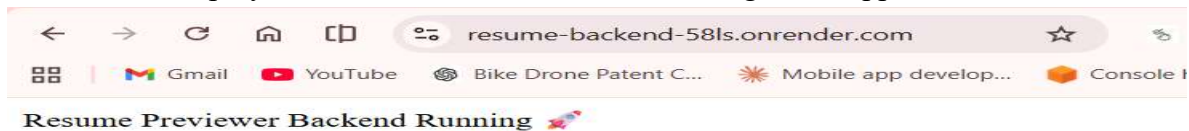


Figure 8.5: Backend Deployment on Render

8.6 Frontend Deployment on Vercel

The frontend application was deployed using Vercel.

Vercel automatically hosted the frontend project from GitHub and provided a live deployment URL.

The frontend deployment enabled users to access the application through a browser interface.

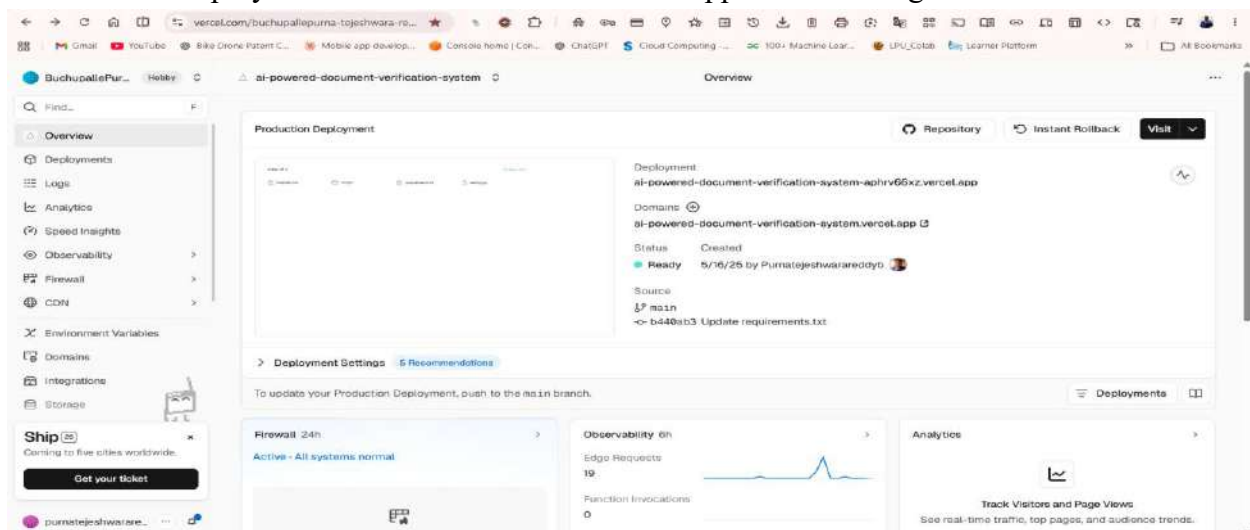


Figure 8.6: Frontend Deployment on Vercel

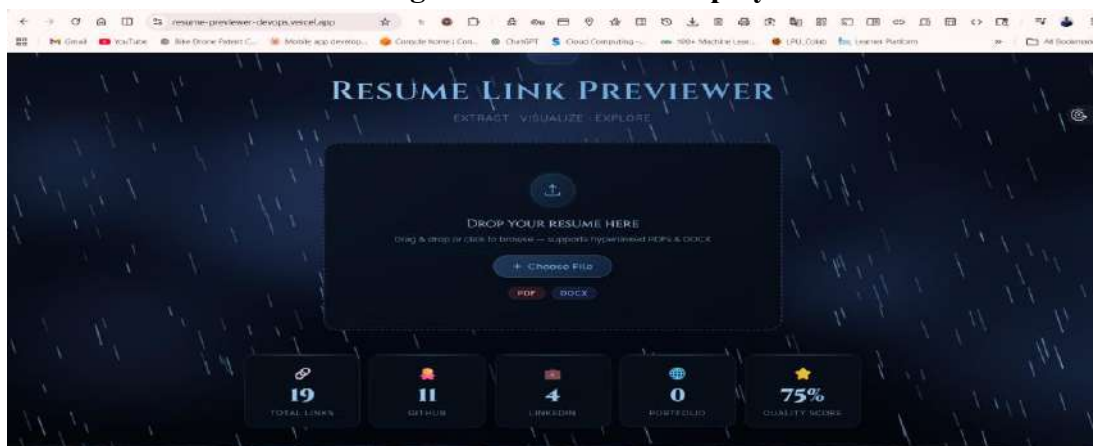


Figure 8.7: Live Frontend Application

8.7 Conversion Plan

The conversion plan describes the transition from local development to cloud deployment.

The following steps were followed:

1. Develop application locally
2. Test frontend and backend locally
3. Create Docker containers
4. Configure Jenkins CI/CD pipeline
5. Push Docker images to Docker Hub
6. Deploy backend on Render
7. Deploy frontend on Vercel
8. Verify live deployment

This process ensured smooth migration from development environment to production environment.

8.8 Post-Implementation and Software Maintenance

After implementation, the application was monitored and tested regularly.

Maintenance activities include:

- ❖ Updating Docker images
- ❖ Monitoring Jenkins pipeline execution
- ❖ Fixing deployment issues
- ❖ Updating project dependencies
- ❖ Maintaining GitHub repository

Future improvements can include:

- ❖ User authentication
- ❖ Resume upload feature
- ❖ Database integration
- ❖ Monitoring dashboards
- ❖ Kubernetes deployment

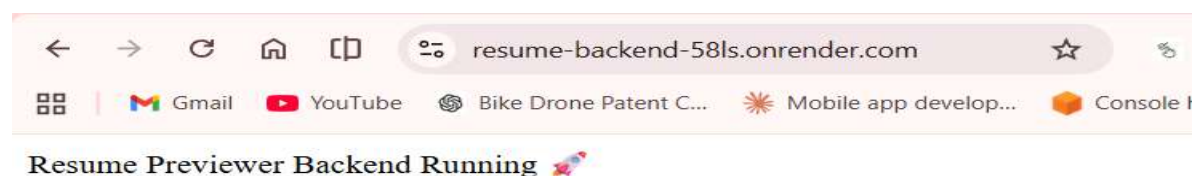


Figure 8.8: Live Backend Service

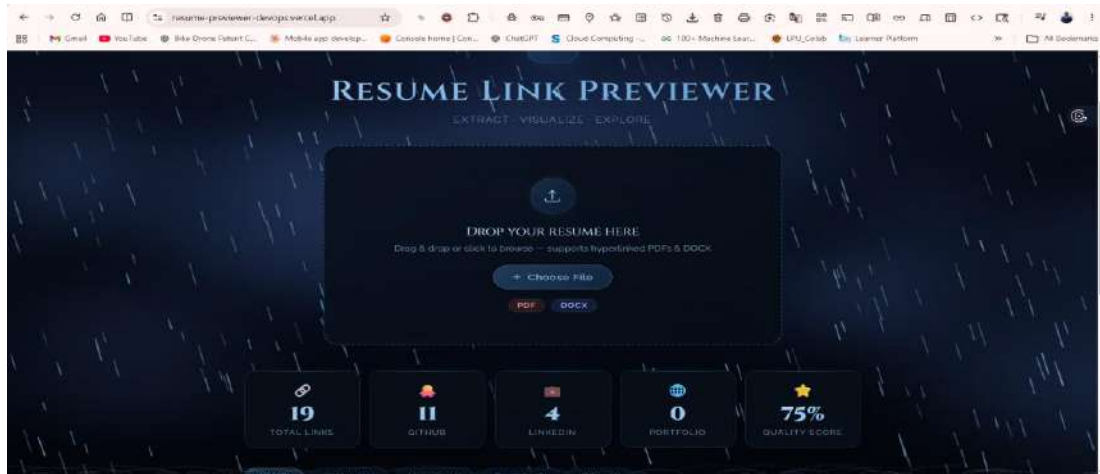


Figure 8.9: Live Frontend Website

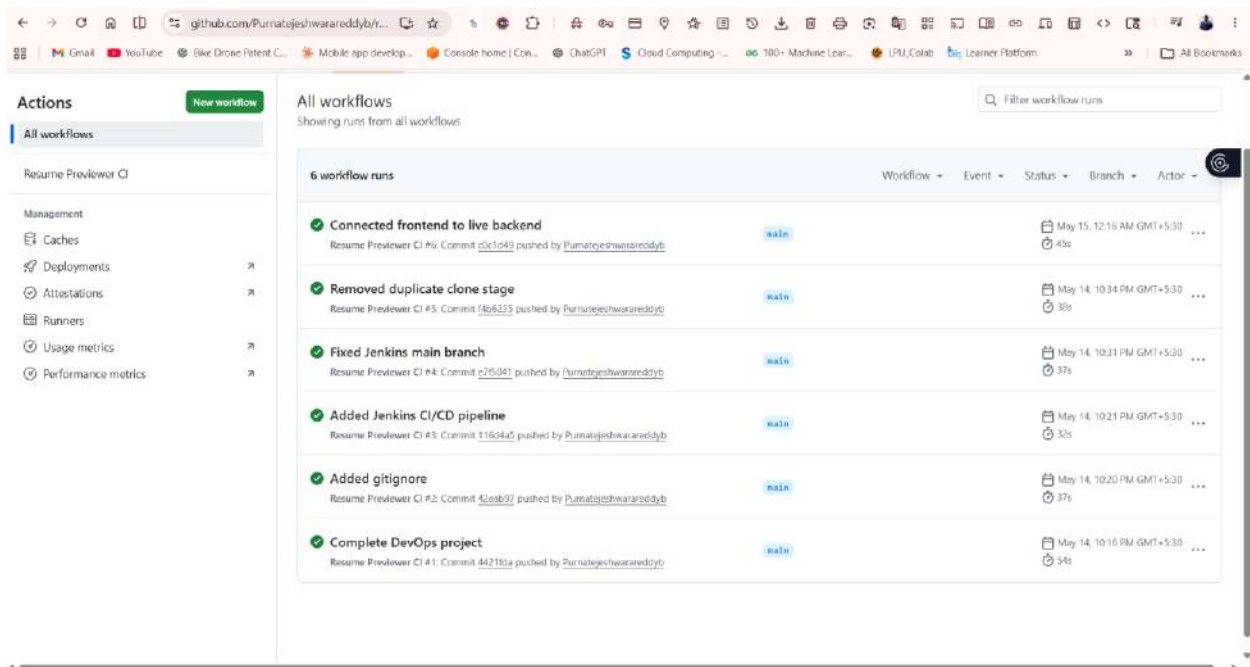


Figure 8.10: Successful CI/CD Deployment

9. Project Legacy

9.1 Current Status of the Project

The Resume Previewer DevOps project has been successfully developed and deployed. The frontend and backend services are working correctly in both local and cloud environments.

The following tasks were completed successfully:

- ❖ Frontend development
- ❖ Backend development
- ❖ Docker containerization
- ❖ Jenkins CI/CD pipeline setup
- ❖ GitHub integration
- ❖ Docker Hub image deployment
- ❖ Backend deployment on Render
- ❖ Frontend deployment on Vercel

The project is currently in a stable working condition.

9.2 Remaining Areas of Concern

Although the project works successfully, there are still some areas that can be improved in the future.

Some remaining concerns include:

- ❖ No database integration
- ❖ No user authentication system
- ❖ Limited frontend features
- ❖ No Kubernetes orchestration
- ❖ Limited monitoring and logging support
- ❖ Free cloud services may have deployment limitations

These improvements can make the system more scalable and production-ready.

9.3 Technical and Managerial Lessons Learnt

During the development of this project, many technical and practical concepts were learned.

Technical Lessons Learnt

- ❖ Understanding Docker containerization
- ❖ Configuring Jenkins pipelines
- ❖ Managing GitHub repositories
- ❖ Deploying applications on cloud platforms
- ❖ Handling CI/CD workflow errors
- ❖ Working with Linux commands and Docker Compose

Managerial Lessons Learnt

- ❖ Importance of project planning
- ❖ Managing deployment workflow
- ❖ Time management during development
- ❖ Problem-solving during debugging
- ❖ Maintaining documentation properly

The project improved both technical knowledge and practical DevOps skills.

9.4 Future Enhancements

The project can be enhanced further by adding advanced features such as:

- ❖ Resume upload functionality
- ❖ PDF preview support
- ❖ User login and authentication
- ❖ Database connectivity
- ❖ Kubernetes deployment
- ❖ Monitoring dashboard integration
- ❖ Automated testing tools
- ❖ HTTPS and security improvements

These enhancements can improve system functionality and real-world usability.

9.5 Expected Outcome

The expected outcome of the project was to create a working DevOps-based application deployment workflow using modern tools and automation practices.

The project successfully achieved the following outcomes:

- ❖ Automated CI/CD deployment
- ❖ Successful Docker containerization
- ❖ Cloud deployment of frontend and backend
- ❖ Faster deployment process
- ❖ Reduced manual configuration effort
- ❖ Better understanding of DevOps practices

The project demonstrates how DevOps tools simplify software deployment and maintenance.

9.6 Project Legacy

The Resume Previewer DevOps project serves as a practical learning project for understanding modern software deployment methods.

The project demonstrates:

- ❖ Continuous Integration and Continuous Deployment
- ❖ Docker-based application deployment
- ❖ Cloud hosting concepts
- ❖ Automation using Jenkins
- ❖ Real-world DevOps workflow

The knowledge and implementation used in this project can be extended to larger software systems in the future.

10. Source Code / System Snapshots

10.1 GitHub Repository Screenshots

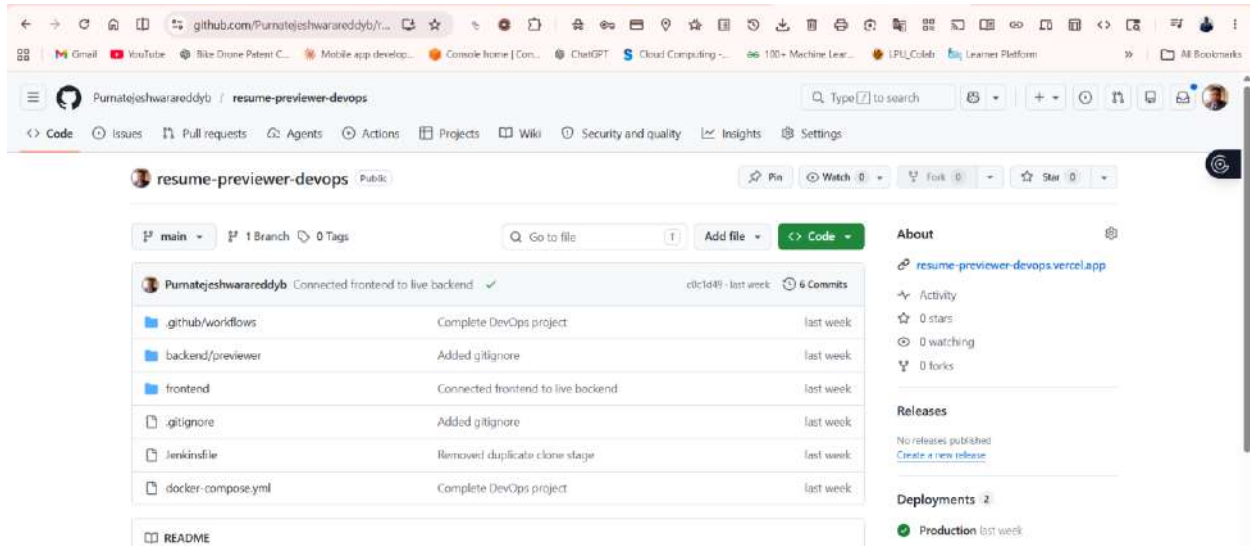


Figure 10.1: GitHub Repository

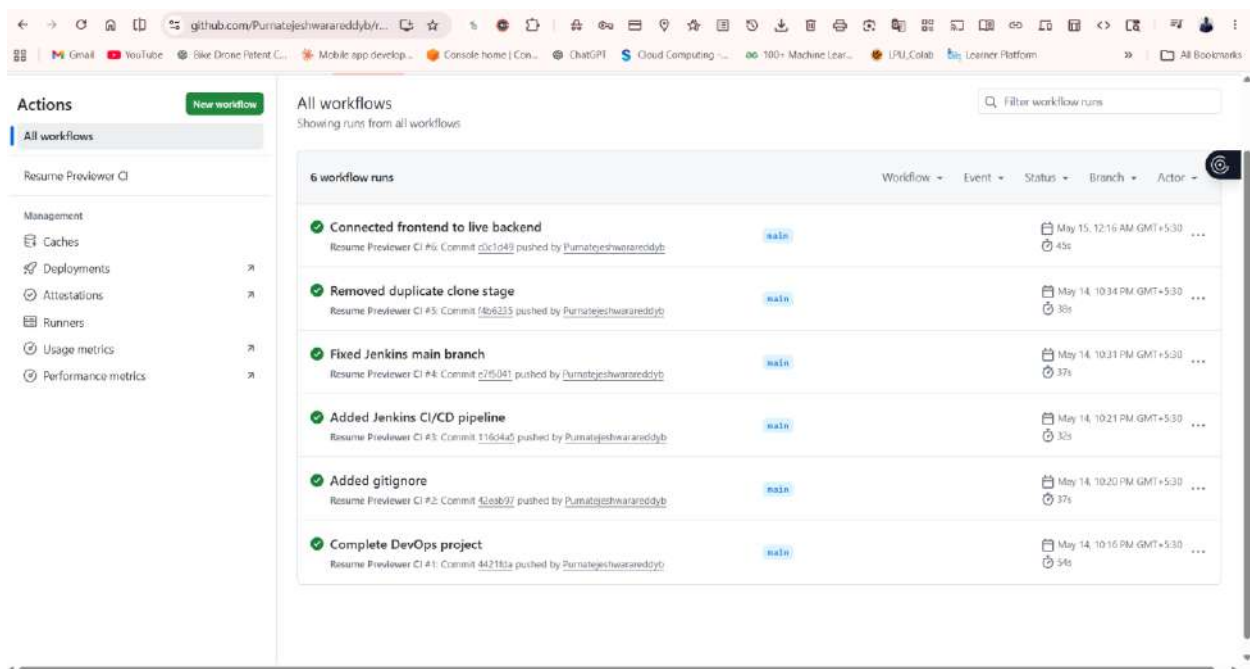


Figure 10.2: Project Source Files

10.2 Jenkins Pipeline Screenshots

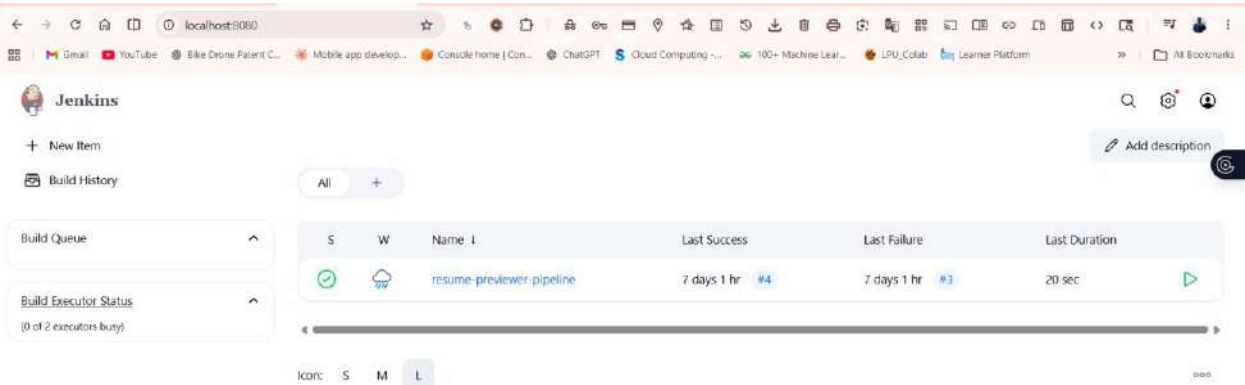


Figure 10.3: Jenkins Dashboard

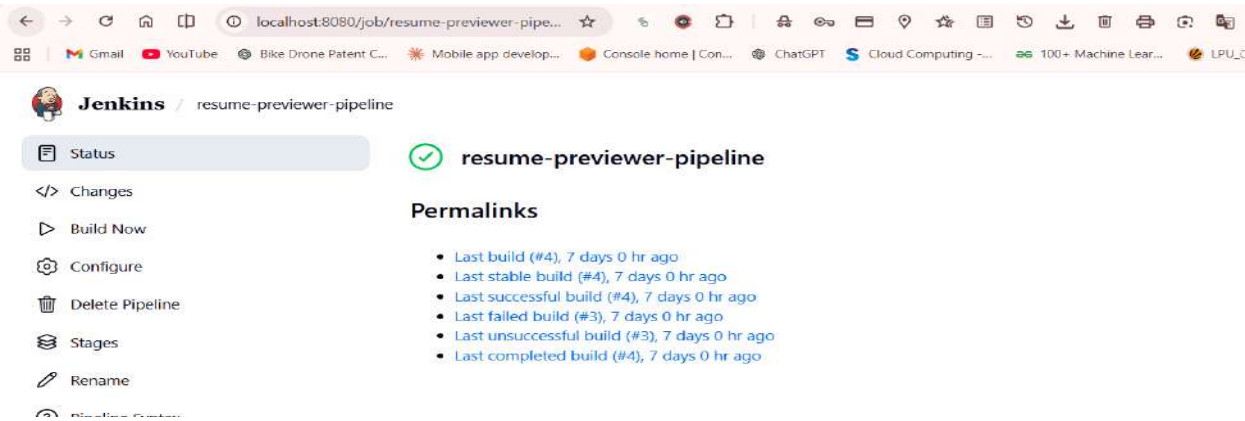


Figure 10.4: Successful Jenkins Pipeline

10.3 Docker Container Screenshots

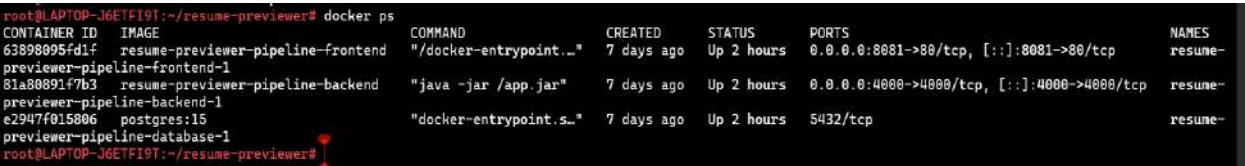


Figure 10.5: Running Docker Containers

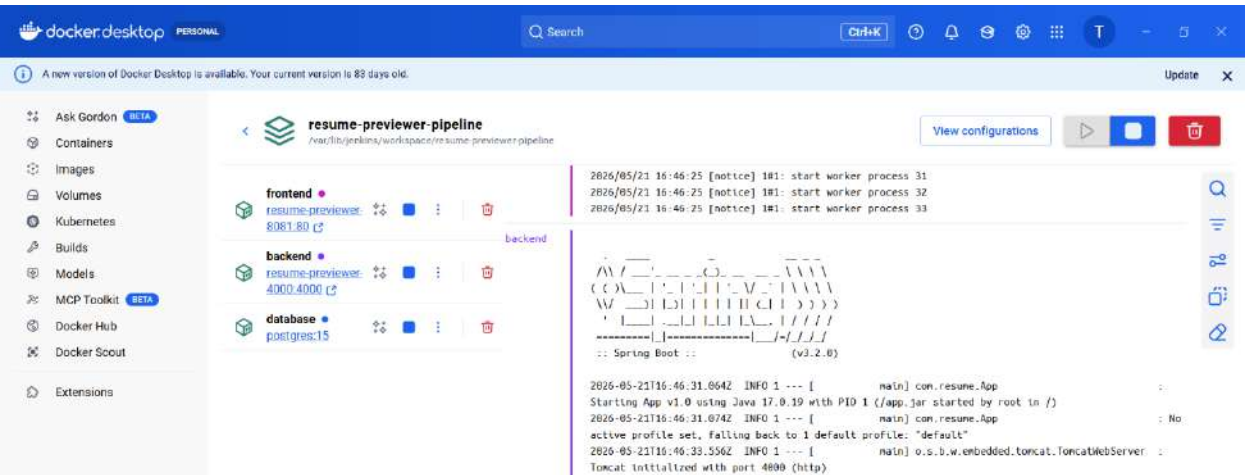


Figure 10.6: Docker Images

10.4 Render Deployment Screenshots

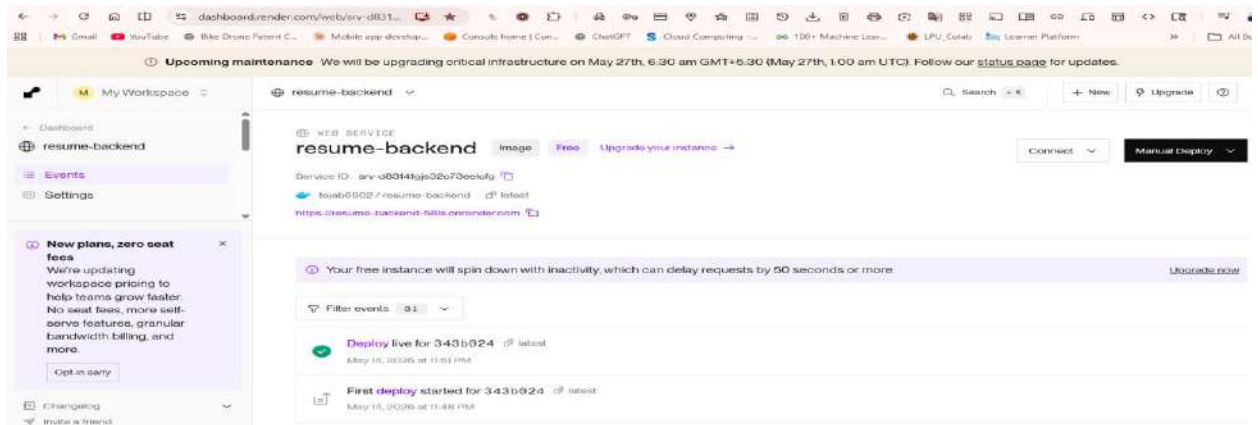


Figure 10.7: Backend Deployment on Render



Figure 10.8: Live Backend Service

10.5 Vercel Deployment Screenshots

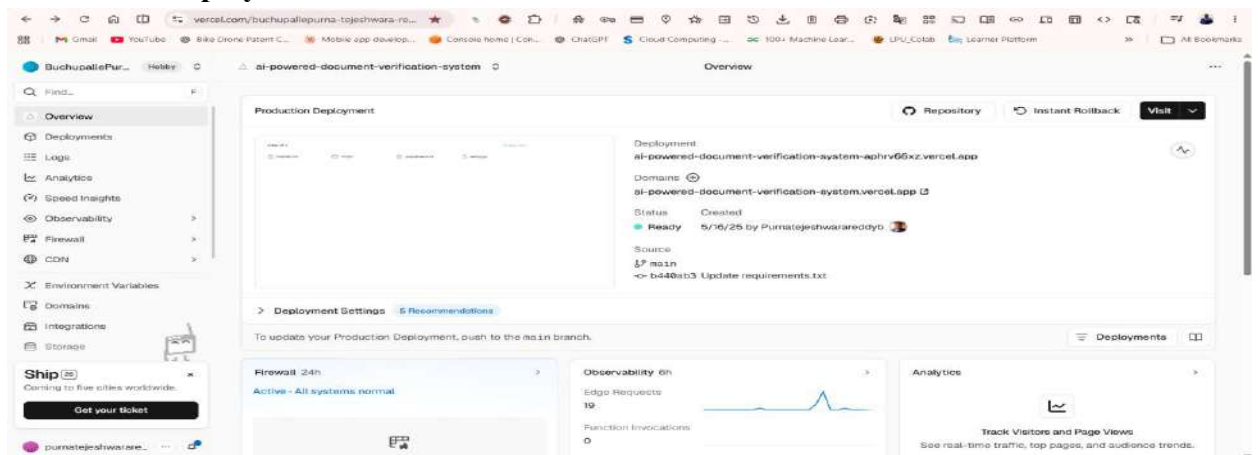


Figure 10.9: Frontend Deployment on Vercel

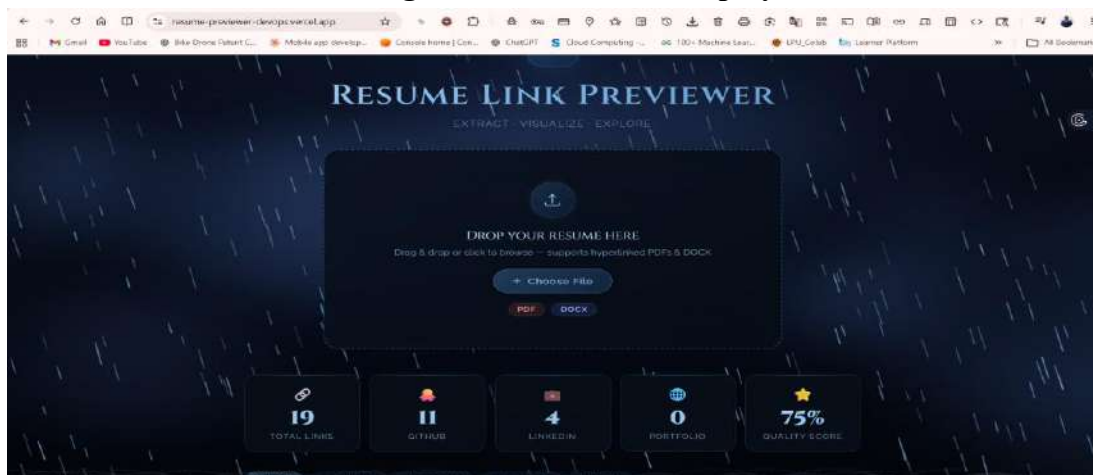


Figure 10.10: Live Frontend Website

10.7 Important Source Code Snippets

```
pipeline {
  agent any

  stages {

    stage('Clone Repository') {
      steps {
        git branch: 'main',
          url: 'https://github.com/Purnatejeshwarareddyb/resume-previewer-devops.git'
      }
    }

    stage('Build Maven Project') {
      steps {
        sh 'cd backend/previewer && mvn clean package'
      }
    }

    stage('Build Docker Containers') {
      steps {
        sh 'docker compose build'
      }
    }

    stage('Deploy Containers') {
      steps {
        sh 'docker compose up -d'
      }
    }
  }

  post {
    success {
      echo 'Pipeline Executed Successfully'
    }

    failure {
      echo 'Pipeline Failed'
    }
  }
}
```

Source Code 10.1: Jenkins Pipeline Code

```
services:

  frontend:
    build: ./frontend
    ports:
      - "8081:80"

  backend:
    build: ./backend/previewer
    ports:
      - "4000:4000"
```

```
database:
  image: postgres:15
  ports:
    - "5432:5432"

environment:
  POSTGRES_DB: resume_db
  POSTGRES_USER: postgres
  POSTGRES_PASSWORD: postgres
```

Source Code 10.2: Docker Compose Configuration

```
FROM eclipse-temurin:17

WORKDIR /app

COPY target/previewer-1.0.jar app.jar

EXPOSE 4000

ENTRYPOINT ["java", "-jar", "app.jar"]
```

Source Code 10.3: Backend Dockerfile

```
FROM nginx:alpine

COPY index.html /usr/share/nginx/html/index.html
```

Source Code 10.4: Frontend Dockerfile

```
name: Resume Previewer CI

on:
  push:
    branches:
      - main

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Set up Java
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: 17

      - name: Build Backend
        run: |
          cd backend/previewer
          mvn clean package
```

Source Code 10.5: Github Actions Workflow File

11. Conclusion

The Resume Previewer DevOps project was successfully developed and deployed using modern DevOps tools and cloud platforms.

The project demonstrated the practical implementation of:

-  CI/CD pipelines
-  Docker containerization
-  GitHub integration
-  Cloud deployment
-  Automated application delivery

The project helped in understanding how automation improves software deployment speed, reliability, and maintainability.

Using Jenkins, Docker, Render, and Vercel simplified the deployment process and reduced manual configuration work. The project also provided practical experience in solving deployment issues and managing real-world DevOps workflows.

Overall, the project successfully achieved its objectives and provided valuable learning experience in DevOps and cloud deployment technologies.

12. Bibliography

1. Docker Official Documentation
2. Jenkins Official Documentation
3. GitHub Documentation
4. Render Deployment Documentation
5. Vercel Official Documentation
6. Spring Boot Documentation
7. Maven Official Documentation
8. Ubuntu Linux Documentation
9. DevOps Learning Resources and Tutorials
10. Online articles and technical blogs related to CI/CD and Docker deployment

Project links

Render Backend Service Dashboard:

<https://dashboard.render.com/web/srv-d83141gjs32c73eelcfg>

Live Backend API Running on Render:

<https://resume-backend-58ls.onrender.com>

Vercel Frontend Deployment Dashboard:

<https://vercel.com/buchupallepurna-tejeshwara-reddys-projects/resume-previewer-devops>

Live Resume Previewer Frontend:

<https://resume-previewer-devops.vercel.app/>

Docker Hub Image Repository:

<https://hub.docker.com/repositories/tejab6902>

Jenkins CI/CD Pipeline Execution:

<http://localhost:8080/job/resume-previewer-pipeline/>

Main GitHub Repository:

<https://github.com/Purnatejeshwarareddyb/resume-previewer->

GitHub Token Configuration for Authentication:

<https://github.com/settings/tokens>

GitHub Actions CI Workflow Execution:

<https://github.com/Purnatejeshwarareddyb/resume-previewer-devops/actions>